

Modular, Full-System Verification

Gregory Malecha
BlueRock Security, Inc
USA
gregory@bluerock.io

Simon Hudon
BlueRock Security, Inc
USA
simon@bluerock.io

Hoang-Hai Dang
BlueRock Security, Inc
Germany
hai@bluerock.io

Jan-Oliver Kaiser
BlueRock Security, Inc
Germany
janno@bluerock.io

Paolo G. Giarrusso
BlueRock Security, Inc
Germany
paolo@bluerock.io

David Swasey*
Riverside Research, Inc
USA
pswasey@riversideresearch.org

ABSTRACT

We present an approach to specifying operating systems that is both highly modular and supports deriving various properties necessary to modern operating systems. The approach combines a machine semantics decomposed by privilege levels with a separation logic specification of the operating system API. We describe how the specification style enables natural proofs of robust safety in addition to deep behavioral refinements of user-mode applications running atop the OS. This approach enables simple and flexible concurrent specifications and unlocks new opportunities for whole-system verification.

KEYWORDS

Operating system specification, Formal verification, Multi-core verification, Separation logic, Iris

ACM Reference Format:

Gregory Malecha, Hoang-Hai Dang, Paolo G. Giarrusso, Simon Hudon, Jan-Oliver Kaiser, and David Swasey. 2025. Modular, Full-System Verification. In *Workshop in Hot Topics in Operating Systems (HOTOS 25)*, May 14–16, 2025, Banff, AB, Canada. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/3713082.3730387>

1 INTRODUCTION

Systems-software verification has been an active area of research for almost two decades. Existing work lays a foundation for reasoning about operating systems (OSes), but much of it treats OSes as fundamentally different than other

pieces of software. The verifications are built around monolithic (whole-program) proof principles [11] or rely on highly stylized coding patterns [8]. They are also limited in their ability to capture concurrency and rely on either completely eliminating it [11] or require all specifications to be linearizable [8]. While these approaches simplify reasoning, they do not account for the complexities of modern systems.

We aim for an operating system *specification* that is

- (1) **Realistic**. Modern OS APIs are rarely atomic, and concurrently executing system calls can lead to subtle behaviors that the specification must account for. These include weak memory behaviors that are pervasive on modern platforms [18, 21, 20].
- (2) **Two-sided** [1]. It must be usable not only to verify the operating system itself, but also to support user-mode reasoning on top of the OS. In particular, clients should be able to prove strong functional correctness properties of user-mode code, including behavioral refinements, using only the OS specification.
- (3) **Modular**. It must scale to support the large and complex APIs present in modern systems. This demands both thread- and component-local specifications that are capable of encapsulating implementation details.
- (4) **Robustly Safe** [7]. It must support running *arbitrary, untrusted* user-mode code and enable user-mode applications to use the OS specification to establish their own integrity even in the presence of (unverified) applications.

To achieve these goals, we have developed a *hybrid specification* approach to specifying OSes. A hybrid specification combines a stylized, low-level, operational processor model (similar to [2]) with separation logic specifications of the privileged behaviors provided by the OS. This approach “passes through” user-mode reasoning principles while allowing concise and modular specifications of OS abstractions. We present a hybrid specification of the NOVA microkernel [22], a multi-core, highly concurrent, capability-based microhypervisor and which constitutes the foundation of

*Work done while at BlueRock Security, Inc.

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

HOTOS 25, May 14–16, 2025, Banff, AB, Canada

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1475-7/25/05.

<https://doi.org/10.1145/3713082.3730387>

the BlueRock™ virtualization stack. We are using this hybrid specification and verifying, on one hand, that NOVA satisfies it and on the other hand that thousands of lines of user-mode libraries and applications built upon that specification in turn each satisfies their own specification. This verification involves proving behavioral refinements of highly concurrent code and requires precise reasoning about the connection between the OS and the architectural semantics.

2 HYBRID OS SPECIFICATIONS

An OS specification must be able to describe the behavior of applications running atop it. Application behavior is largely dictated by the underlying hardware architecture, but with “privileged” logic provided by the OS rather than directly by the hardware. For example, while the architecture describes the behavior on a page fault in terms of low-level concepts (interrupts are disabled, certain registers are saved, etc), the OS describes this behavior to applications in terms of OS concepts, *e.g.*, NOVA will block the execution context (thread) on the page fault portal. Hybrid specifications capture this setup by using a (operational) CPU model that has been decomposed into privilege levels (§2.1). The hybrid specification retains the unprivileged portions but replaces the privileged parts with OS-logical state (§2.2) and weakest pre-conditions expressed in separation logic (§2.3–§2.4).

2.1 Decomposing Machine Semantics

To specify the system behaviors modularly, we build on a model that captures the machine semantics as a *distributed system of hardware components* including processors (CPUs), memory, interrupt and bus controllers, and (peripheral) devices [19, 14]—each component as a labeled transition system (LTS). The distributed-system model formally captures the separation of individual components and enables us to build the full machine model compositionally, incrementally, and in an open-ended way—for example, enabling reasoning about different machine configurations.

Within the machine model, the processor semantics must be further decomposed into different privilege levels. This decomposition is finer-grained than the “exception levels” of modern processors and is derived from the reachability of processor state. Executing a single instruction in a modern processor requires multiple steps and uses information from multiple protection levels. For example, reading the instruction pointer is an unprivileged operation, but translating a virtual address is privileged because it requires reading control registers, TLB state, and page tables. Additionally, interactions with putatively unprivileged state that can be intercepted by the OS, *e.g.*, when the kernel disables FPU access, or when it traps register writes, must also be privileged.

The hybrid specification approach requires that the architecture semantics expresses the decomposition using an unprivileged LTS with a handler. Unprivileged steps are codified as silent steps (τ), while accesses to privileged state are exposed through events that are “handled” using the privileged state. The handler can return the value of privileged registers or signal faults back to the unprivileged state. **This event and handler architecture drives the setup of hybrid specifications.**

This separation of privileged and unprivileged state is driven by the architecture and can therefore be reused by different OS specifications. Several threads of current work [6] have investigated this decomposition on the ARM architecture, and preliminary investigation suggests that a similar decomposition likely holds for x86. This is not surprising given the close connection between this decomposition and Popek and Goldberg’s virtualizability requirements [16].

Challenges. While sketches of this decomposition exist for fragments of ARM, there is not yet a complete decomposition, let alone one that is proven with respect to an authoritative semantics. One challenge is simply the size of architectural specifications. Beyond the size, this task is complicated by instruction scheduling and the weak (memory) semantics common in modern architectures [18, 21]. Specifically, inter-instruction scheduling means that some interactions between the privileged and unprivileged layers must be non-atomic, though operational models suggest that these operations can be linearized using finer-grained steps.

2.2 Specifying OS State in Separation Logic

At a high level, the state of an OS is effectively a collection of separate objects. In NOVA, these objects include protection domains, execution contexts, scheduling contexts, semaphores, and portals. In mainstream OSes, these include inodes, file descriptor tables, and address spaces. While finite maps—from object identifiers to their state—provide a good model for this form of state, their global nature, *i.e.*, *a finite map carries information about many objects*, is devastating to modularity. Separation logic [3, 10] can be used to decompose such a global map to achieve modularity.

The hybrid specification eschews the concrete model of the OS state and instead exposes it as modular separation logic *ghost state*, a feature common in modern separation logics. For example, rather than exposing a finite map from semaphore names to their value and wait queue, the NOVA specification exposes two *separate* predicates, $sm \xrightarrow{\text{val}} n$ and $sm \xrightarrow{\text{queue}} \text{ecs}$, that expose this data *for each* semaphore sm . While, the difference seems purely aesthetic, the locality provides multiple practical benefits: (1) a client working with the semaphore sm does not need to be concerned about the state of other semaphores or objects; (2) the other objects

can be used and reasoned about independently by other concurrent clients; and (3) as the specification designer, we can incrementally refine the specification by adding new state without invalidating clients, which crucially reduces frictions in evolving the OS and its specification.

Beyond the conciseness and expressivity, another benefit of directly using separation logic is that we simply do not need to construct the OS model, which is often a substantial upfront cost. Instead, we incrementally extend the separation logic specification by exposing more state and using it to refine the OS behaviors. This approach allows us to co-develop, and maintain, user-mode proofs that use the NOVA specification, crucial work for realizing our ultimate goal of full-system verification.

2.3 Specifying Privileged Behavior

With the state expressed in separation logic, we now leverage separation logic as an expressive, higher-order programming language to describe the behavior of privileged steps. We capture the behaviors as updates to the resources representing the state: the step consumes a resource—the pre-condition—and produces a new resource—the post-condition. For example, we can write the specification for the CTRL_UP step of a semaphore as

$$\text{do_up } sm \ K := \exists n, sm \xrightarrow{\text{val}} n * (sm \xrightarrow{\text{val}} n+1 \multimap K).$$

This assertion captures a *non-atomic* increment of the semaphore value, with the pre-condition $sm \xrightarrow{\text{val}} n$ for some client-picked n and the post-condition $sm \xrightarrow{\text{val}} n+1$ with a continuation K . This idiomatic use of separating conjunction ($*$) and separating implication ($\multimap$) obliges clients to prove that: (1) they have (logical) ownership to establish the pre-condition $sm \xrightarrow{\text{val}} n$, and (2) they can prove K when given the updated state, $sm \xrightarrow{\text{val}} n+1$.

Note that do_up itself is also a separation logic resource, and so we can chain multiple resources together to specify multiple steps of the OS. Figure 1 expresses the semantics of the multi-step semaphore up hypercall which resolves the selector (and checks permissions) and then updates the semaphore value. It uses $\text{resolve_sel } sel$ which specifies the selector resolution in a similar way to do_up using the ownership $sel \xrightarrow{\text{cap}}_q^o c$ of the capability to the targeted selector sel . If the selector resolution succeeds, the rest of the code performs the permission check ($\text{perms} \in \text{SM_UP}$) and the actual increment ($\text{do_up } sm$). Otherwise, BAD_CAP is returned. Crucially, the use of separation logic enables this specification to be completely independent of other NOVA features.

2.4 Supporting Concurrency

The specification style in §2.3 is sequential: it requires the client to give up exclusive ownership of the semaphore for

```
ctrl_sm.up_spec sel K :=
  (* LP: resolve the selector to a kernel object *)
  if resolve_sel sel = Some (sm, perms) then
    (* LP: increase the semaphore value. *)
    if perms ∈ SM_UP then do_up sm K else K BAD_CAP
  else K BAD_CAP (* return BAD_CAP if sel is unmapped *)
```

Figure 1: The (simplified) NOVA specification of CTRL_UP chains linearization points (LPs) for checking permission and increasing the semaphore value.

the whole hypercall. To make this pattern applicable in concurrent settings, we convert it to use Iris’s *atomic updates* (AUs) [9, 10].

$$\text{AU } \ll \exists x, P(x) \gg \mathcal{E}_1, \mathcal{E}_2 \ll \forall y, Q(x, y), \text{ COMM } R(x, y) \gg$$

This form specifies a *logically atomic* update from the resources $P(x)$ (the atomic pre-condition) to $Q(x, y)$ (the atomic post-condition), where x are input arguments and y are returned results. After the update, the non-atomic post-condition $R(x, y)$ captures the continuation obligation (analogous to K). While the technical construction is subtle, it suffices to see AUs as a first-class representation of linearization points (LPs) as resources. This allows us to chain AUs together to specify multi-step hypercalls with multiple LPs. Concretely, we can keep the structure of the specification in Figure 1, and convert each step specification to AUs.

$$\text{do_up } sm \ K := \text{AU } \ll \exists n, sm \xrightarrow{\text{val}} n \gg \top, \mathcal{E}_{\text{nova}} \ll sm \xrightarrow{\text{val}} n+1, \text{ COMM } K \gg$$

The final ingredient to programming with concurrent separation logics (CSLs) is the thread-pool semantics, which is typically expressed as the separating conjunction of *weakest pre-conditions* (WPs, see more in §2.5), one per thread. In essence, the weakest pre-condition $wp \ e_1$ in CSLs captures the semantics of a thread e_1 even with concurrent interference of other threads, and so the separating conjunction $wp \ e_1 * wp \ e_2$ captures all interleaving behaviors of e_1 and e_2 running concurrently together. The expressivity can hardly be overstated: this separating conjunction succinctly captures the crux of concurrent execution and, when coupled with atomic updates, converts a naïve, sequential OS specification into a concurrent one. The product is a highly modular, thread-local reasoning principle—*i.e.*, we can reason about each $wp \ e$ in isolation.

2.5 The Hybrid Specification

The hybrid specification combines the unprivileged operational semantics (§2.1) with the separation logic specification for privileged operations (§2.3–§2.4) using a notion of weakest pre-condition for OS threads. In NOVA, $wp_{\text{nova_ec}}$ captures the behavior of a NOVA execution context ec .

```

Lemma wp_nova_ec_intro ec :
  |={T, E_nova}>= >
  ∃ regs k, ec.regs ec 1 regs * ec.kstate ec 1 k *
  (∀ regs' evt,
    [ cpu_step regs evt regs' ] -* ec.regs ec 1 regs' -*
    match evt with (* The NOVA "handler" *)
    | τ => ec.kstate ec 1 kont.ipc_tip -*
      |={E_nova, T}>= wp_nova_ec ec
    | syscall call_id =>
      wp_hypercall call_id (|={E_nova, T}>= wp_nova_ec ec)
    | trap => wp_handle_trap (|={E_nova, T}>= wp_nova_ec ec)
    | mem e => wp_mem e (|={E_nova, T}>= wp_nova_ec ec)
    | ... (* other "event handlers" *) ...
  )
  end)
  ⊢ wp_nova_ec ec.

```

Figure 2: wp_nova_ec ec describes how the unprivileged EC state drives the user-thread’s behaviors and how NOVA handles privileged events from the thread.

The lemma `wp_nova_ec_intro` (Figure 2) captures the reasoning principle for execution contexts in two parts. Control is driven by the decomposed semantics (`cpu_step regs evt regs'`, §2.1) where `regs` is the current unprivileged state of the execution context (captured by the separation logic assertion `ec.regs ec 1 regs`). The `match` “handles the privileged event” (`evt`) by interpreting it as a weakest precondition. In the τ case (when there is no effect on the privileged state), control immediately returns to `wp_nova_ec` which executes the next step. The `syscall` case delegates to the hypercall WPs such as for `ctrl_sm` (Figure 1). The `trap` case describes the behavior on user-mode faults such as page faults or floating point exceptions, which, in NOVA, block execution on architectural portals. The `mem` case captures the behavior on memory accesses which consist of two parts: (1) the virtual address translation uses privileged OS state reflected in separation logic—that state encapsulates the MMU hardware and the in-memory, second-stage, page tables; and (2) if the translation is successful, the actual memory access is captured using an AU that accesses the memory state.

Startup. Each WP of an OS thread captures the steady-state behavior of the machine running the thread. Eventually, this behavior must be connected to the OS startup code. Hybrid specifications express the top-level specification as a higher-order separation logic formula.

```

Theorem os_ok :
  (OS-level startup resources -* wp_thread boot_thread)
  ⊢ (machine-level boot resources -* wp_machine).

```

`os_ok` connects the machine-level resources in the conclusion to the OS-level resources in the premise. The machine-level boot resources will include resources such as the raw contents of memory as well as the privileged state of the processor. The OS-level resources will reflect the initial state when

the OS launches the first thread, *e.g.*, the processor’s logical view of its memory and the unprivileged register state. The power of this specification lies in converting machine-level proof obligations into OS-level obligations. In §3, we will see how this specification enables whole-system reasoning.

Challenges. The hybrid specification approach leverages the power of separation logic and connects it to the abstractions present in architectural models. Beyond the decomposition challenges mentioned in §2.1, another major challenge to OS specification is weak memory, which significantly complicates specifications when kernel code interacts with user-accessible memory. The NOVA specification defers weak memory interactions to the “instruction scheduler” of the memory model (as in [18]), which is explicit in the hybrid specification. However, whether this is sufficient to capture *all* of the behaviors that might arise when speculating across the user- and kernel-mode boundary remains to be proven.

3 USING THE HYBRID SPECIFICATION

We now turn to the use of hybrid specifications to establish properties of larger systems that include *both verified and unverified* code. We note that many of the benefits here are “simply” inherited from the expressivity of modern CSLs (such as Iris), but the ability to exploit these features pervasively throughout verification is very valuable.

3.1 Proving Robustness

One crucial feature of operating systems is that they should be able to run *arbitrary* programs. The use of WPs within hybrid specifications makes proving this non-trivial as it places a proof obligation on the user. To understand the problem, suppose that the NOVA specification required clients to prove that permission checks would succeed. Such a specification would justify removing permission checks in the implementation and would still allow verifying many applications, but it would not be sufficient to show that *all* programs can safely run on NOVA.

Establishing robustness for a hybrid specification requires that we prove that the user-mode obligation (premise of `os_ok`) is satisfiable for *an arbitrary user-mode program*. Fortunately, the intuitions from proving semantic type soundness [26] provide an easy way to prove this directly using the hybrid specification. Effectively, we bundle all of the kernel resources into an invariant that existentially quantifies all of the uninteresting (*low integrity*) values. For example, we need to know that: every execution context has registers, every mapped selector references a valid object, every mapped address is backed by actual physical memory, and so on. More importantly, we do not care what values the registers have, what object is referenced, or what memory provides the backing.

Using this “robust-safety” invariant, we can prove `wp_thread` for the arbitrary user code. The obligations reduce to two cases. For pure steps in the operational part of the hybrid WP, the proof is trivial because we know that the invariant contains some register resources for the currently running thread. After the step, the register file may have been updated, but its contents are existentially quantified so it is trivial to re-establish the invariant. For a privileged step, it suffices to show that the invariant implies the WP. While we have not yet carried out this proof formally, paper-pencil proofs here seem straight-forward since ownership of all reachable references are maintained in the invariant.

Fine-grained Robust Safety. Robustness shows that the kernel is protected from applications, but it is too weak to show that applications can be protected from each other. This richer isolation property is crucial to establish separation properties, but establishing separation purely at the OS level is often too weak because *any* sharing often breaks the property. For example, microkernel-based systems such as Genode [15] rely on a user-mode “master controller” to aid the hypervisor in resource management. Similarly, in cloud environments, user-mode applications orchestrate containers and virtual machines. In these situations, it is not the OS alone that provides the isolation, but rather the combination of the OS and some applications.

Fortunately, not all is lost. Separation logic makes it is easy to establish *separate invariants* for different groups of resources and prove each one supports arbitrary code. These invariants are often more nuanced than the simple robust safety invariant above. For example, they can allow processes to share resources in a controlled way, such as read-only memory or sharing only the `CTRL_UP` permission on a semaphore. These sharing strategies can be highly application-specific, but the ability to establish their robustness within separation logic enables developers to carry out these proofs straightforwardly.

3.2 Assembling a Whole System Proof

Robustness provides a strong property for the OS in isolation, but ultimately, we aim for stronger properties about the *behavior of the full system* where the OS is just a single component. Since systems are interactive, we aim to prove *behavioral refinements* that establish that all possible behaviors of the system are also behaviors of a specification. We aim to produce these proofs by assembling the verifications of applications carried out on top of the OS specification, and the verification of the OS itself against such specification.

While the details are somewhat nuanced, the high level picture is simple. We decompose the correctness condition by each level of abstraction, *i.e.*, machine, OS, and application levels, and state the condition of each level as an idiomatic

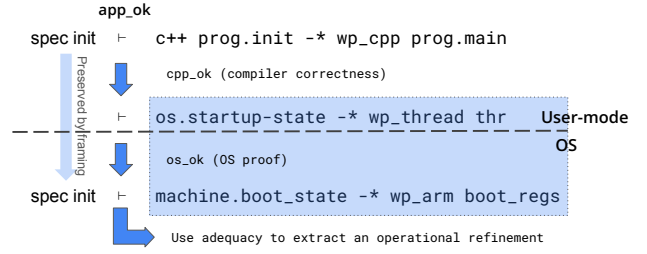


Figure 3: Building a full system proof using an OS proof against a hybrid specification.

specification in the form of `os_ok`. We then use framing to transport the refinement from the machine level to the application level. Figure 3 sketches how we plan to achieve this following the CaReSL technique [27] for proving behavioral refinements in separation logic. To prove that the whole implementation refines the behaviors of a specification system (here, the specification of the application), we embed the specification system into separation logic and prove that the behaviors of the whole implementation (here, the observable behaviors of the system level) are allowed by the specification. This end-to-end proof starts by requiring the proof `app_ok` of the application on top of the OS in Figure 3: assuming the specification system and the application are in their initial states (`spec init` and `c++ prog.init`) it is safe to execute the application program entry point (`wp_cpp prog.main`).

We can then use the compiler correctness theorem (`cpp_ok` in the example) to lower the WP of the application’s programming language (C++) to the OS level. We now have the OS-level WP (e.g., `wp_nova_ec`) which embeds the user-mode semantics of the application. This step, following `wp_nova_ec`, decomposes into two parts. The unprivileged steps are passed through from the architecture semantics, while privileged behaviors require the compiler’s reasoning about the OS abstractions. While we have not carried out these proofs formally yet, we believe that they are formalizable by leveraging compiler correctness proofs such as those in Simuliris [5].

The last step is linking the user-mode proof to the OS proof, which is performed using the higher-order specification of the OS (`os_ok`). The crucial piece in the whole proof is that the `spec init` resource is preserved by framing (unaffected by the proof steps). This leads us to the machine-level proof at the bottom of Figure 3. At this level, assuming an appropriate operational model of the physical machine,¹ we can use a standard CSL adequacy result to extract an end-to-end operational refinement.

¹Operational models that treat (some of) the intricacies of architectural details do exist [2, 17, 4], but more work is necessary to model all of the systems features that are necessary to support a real OS.

3.3 Program Verification using the Spec

To date, we have carried out several (on-going) verifications of user-mode libraries and applications, and aim to ultimately verify a full virtualization stack atop the NOVA specification. These are verifications for tens of thousands of lines of C++ implementations, including a virtual network switch, a virtual machine manager, a virtual console multiplexer [13], several virtual devices, and user-mode libraries wrapping NOVA’s primitives. We carry out the proofs in the Rocq proof assistant² [25] using the BRiCk program logic [24, 12]. BRiCk supports a large fragment of modern, concurrent systems programming C++. The proofs are performed with by generic separation logic automation which dramatically accelerates the verification time and has enabled us to scale our verification to large and complex code bases.

Verifying Semaphores. To demonstrate the two-sided nature of hybrid specifications, we verified a C++ semaphore library against the NOVA semaphore specification and also verified the NOVA implementation of semaphores.³ The user-mode semaphore implementation is based on the implementation described in [23] and uses a user-mode atomic to reduce kernel contention. On top of this core semaphore, the library provides implementations of mutexes, reader-writer locks, and the C++ lock guard idiom. On the NOVA side, the proof covers the implementation of `Sm::up`, `Sm::dn`, and the timeout queues, but does not yet cover capability resolution or the interaction with the scheduler. Both of these proofs rely on fine-grained concurrent operations and feature higher-order reasoning patterns to make the specifications and proofs highly reusable. A current limitation is that all of the proofs are about the C++ source code, and we have not verified the assembly glue code that invokes NOVA (from user-mode) or the NOVA kernel-entry and -exit logic.

Verifying a Virtual Machine Monitor. The largest verification effort that we have undertaken to date (but not yet completed) is the verification of a virtual machine monitor (VMM). The goal is a behavioral refinement between the VMM’s C++ implementation and a distributed-systems-based specification model of a physical computer with multiple CPUs, memory, busses, and devices. This property is an instance of the end-to-end refinement in Figure 3, *i.e.*, it states that any behavior of the VMM running a particular guest image is also a behavior of the specification model (with a particular hardware configuration). We found the expressivity of separation logic naturally fits the distributed system specification of guest hardware, allowing our proof to closely match the architecture of the code and the specification.

²Previously called the Coq proof assistant.

³Our proof does not yet handle interrupt-bound semaphores and is based on slightly simplified code that uses sequentially consistent atomics.

Of particular note is the way that NOVA’s hybrid specification enables us to reason about hardware virtualization. The VMM implementation uses NOVA’s support for virtual CPUs (execution contexts) to execute the guest and its architectural portal mechanism to handle faults. The data abort logic that tracks the guest state on a faulting memory access through the fault handler specification and into the C++ implementation constitutes a key portion of the proof. The C++ code handles the fault, often by invoking a virtual device indirectly through the virtual bus, re-establishes the invariant necessary for hardware virtualization, and returns to NOVA, which re-configures the virtual CPU and resumes its execution. This control path is highly complex, going back and forth between virtual CPU code, the NOVA implementation, and the C++ code of the VMM, but the unifying formalism of separation logic and the hybrid specification of NOVA make such reasoning relatively natural.

4 CONCLUSIONS

We described an approach to full system verification using a hybrid operational and axiomatic specification for operating systems. This style combines an operational characterization of user-mode code with a separation logic specification of the OS. Hybrid specifications build on decomposed architectural models (§2.1) that appear promising but require more investigation. The strategy of introducing separation logic early and staying within it both for specification and verification leads to natural specifications and elegant proofs of even sophisticated properties. These properties include both robustness properties which hold for *arbitrary* programs (§3.1) as well as stronger behavioral refinements for concrete (verified) programs (§3.3). To demonstrate the synergies enabled by separation logic, we used the NOVA specification to verify a user-mode semaphore library on top of NOVA semaphores as well as connecting the hardware virtualization specifications exposed by NOVA to the verification of a user-mode VMM running on top of NOVA.

We believe that separation logic can be used by OS developers as a specification language for highly-concurrent systems including operating systems. This approach scales to complex semantics in a highly modular way which suggests that it is likely scalable to larger systems as well. Beyond scaling up, we discussed several future directions related to formalization of the architectural decomposition (especially in connection with weak memory semantics).

ACKNOWLEDGMENTS

Many thanks to Gordon Stewart, Abhishek Anand, Rodolphe Lepigre, Jasper Haag, František Farka, and Yoichi Hirai who contributed to the evaluation of these techniques during their time at BlueRock Security, Inc.

REFERENCES

- [1] Andrew W. Appel, Lennart Beringer, Adam Chlipala, Benjamin C. Pierce, Zhong Shao, Stephanie Weirich, and Steve Zdancewic. 2017. Position paper: the science of deep specification. *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences* 375, 2104 (October 2017).
- [2] Alasdair Armstrong, Thomas Bauereiss, Brian Campbell, Shaked Flur, Kathryn E. Gray, Prashanth Mundkur, Robert M. Norton, Christopher Pulte, Alastair Reid, Peter Sewell, Ian Stark, and Mark Wassell. 2018. Detailed Models of Instruction Set Architectures: From Pseudocode to Formal Semantics. In *Proc. Automated Reasoning Workshop*. 23–24. Two-page abstract.
- [3] Cristiano Calcagno, Peter W. O’Hearn, and Hongseok Yang. 2007. Local Action and Abstract Separation Logic. In *22nd IEEE Symposium on Logic in Computer Science (LICS 2007), 10–12 July 2007, Wrocław, Poland, Proceedings*. IEEE Computer Society, 366–378. <https://doi.org/10.1109/LICS.2007.30>
- [4] Shaked Flur, Kathryn E. Gray, Christopher Pulte, Susmit Sarkar, Ali Sezgin, Luc Maranget, Will Deacon, and Peter Sewell. 2016. Modelling the ARMv8 architecture, operationally: concurrency and ISA. In *Proceedings of the 43rd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (St. Petersburg, FL, USA)*. 608–621. <https://doi.org/10.1145/2837614.2837615>
- [5] Lennard Gäher, Michael Sammler, Simon Spies, Ralf Jung, Hoang-Hai Dang, Robbert Krebbers, Jeehoon Kang, and Derek Dreyer. 2022. Simuliris: a separation logic framework for verifying concurrent program optimizations. *Proc. ACM Program. Lang.* 6, POPL, Article 28 (Jan. 2022), 31 pages. <https://doi.org/10.1145/3498689>
- [6] Paolo G. Giarrusso, Abhishek Anand, Gregory Malecha, František Farka, and Hoang-Hai Dang. 2024. Modularizing CPU Semantics for Virtualization, Talk at PriSC 2024. <https://popl24.sigplan.org/details/prisc-2024-papers/7/Modularizing-CPU-Semantics-for-Virtualization>
- [7] Andrew D. Gordon and Alan Jeffrey. 2001. Authenticity by Typing for Security Protocols. In *14th IEEE Computer Security Foundations Workshop (CSFW-14 2001), 11–13 June 2001, Cape Breton, Nova Scotia, Canada*. IEEE Computer Society, 145–159. <https://doi.org/10.1109/CSFW.2001.930143>
- [8] Ronghui Gu, Zhong Shao, Hao Chen, Xiongnan Wu, Jieung Kim, Vilhelm Sjöberg, and David Costanzo. 2016. CertiKOS: an extensible architecture for building certified concurrent OS kernels. In *Proceedings of the 12th USENIX Conference on Operating Systems Design and Implementation (Savannah, GA, USA) (OSDI’16)*. USENIX Association, USA, 653–669.
- [9] Ralf Jung, Robbert Krebbers, Jacques-Henri Jourdan, Ales Bizjak, Lars Birkedal, and Derek Dreyer. 2018. Iris from the ground up: A modular foundation for higher-order concurrent separation logic. *J. Funct. Program.* 28 (2018), e20. <https://doi.org/10.1017/S0956796818000151>
- [10] Ralf Jung, David Swasey, Filip Sieczkowski, Kasper Svendsen, Aaron Turon, Lars Birkedal, and Derek Dreyer. 2015. Iris: Monoids and Invariants as an Orthogonal Basis for Concurrent Reasoning. In *Proceedings of the 42nd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2015, Mumbai, India, January 15–17, 2015*. ACM, 637–650. <https://doi.org/10.1145/2676726.2676980>
- [11] Gerwin Klein, Kevin Elphinstone, Gernot Heiser, June Andronick, David Cock, Philip Derrin, Dhammika Elkaduwe, Kai Engelhardt, Rafal Kolanski, Michael Norrish, Thomas Sewell, Harvey Tuch, and Simon Winwood. 2009. seL4: formal verification of an OS kernel. In *Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles (Big Sky, Montana, USA) (SOSP ’09)*. Association for Computing Machinery, New York, NY, USA, 207–220. <https://doi.org/10.1145/1629575.1629596>
- [12] Gregory Malecha, Abhishek Anand, and Gordon Stewart. 2020. Towards an Axiomatic Basis for C++. In *The 11th Coq Workshop* (online).
- [13] Gregory Malecha, Gordon Stewart, Frantisek Farka, Jasper Haag, and Yoichi Hirai. 2022. Developing With Formal Methods at BedRock Systems, Inc. *IEEE Security and Privacy* 20, 3 (2022), 33–42. <https://doi.org/10.1109/MSEC.2022.3158196>
- [14] Jeffrey C. Mogul, Andrew Baumann, Timothy Roscoe, and Livio Soares. 2011. Mind the Gap: Reconnecting Architecture and OS Research. In *13th Workshop on Hot Topics in Operating Systems, HotOS XIII, Napa, California, USA, May 9–11, 2011*, Matt Welsh (Ed.). USENIX Association. <https://www.usenix.org/conference/hotosxiii/mind-gap-reconnecting-architecture-and-os-research>
- [15] Norman Feske. [n.d.]. *Introducing Genode*. Genode Labs. <https://genode-labs.com/publications/nfeske-genode-fosdem-2012-02.pdf>
- [16] Gerald J. Popek and Robert P. Goldberg. 1974. Formal requirements for virtualizable third generation architectures. *Commun. ACM* 17, 7 (July 1974), 412–421. <https://doi.org/10.1145/361011.361073>
- [17] Christopher Pulte, Shaked Flur, Will Deacon, Jon French, Susmit Sarkar, and Peter Sewell. 2017. Simplifying ARM Concurrency: Multicopy-Atomic Axiomatic and Operational Models for ARMv8. *Proc. ACM Program. Lang.* 2, POPL, Article 19 (dec 2017), 29 pages. <https://doi.org/10.1145/3158107>
- [18] Christopher Pulte, Shaked Flur, Will Deacon, Jon French, Susmit Sarkar, and Peter Sewell. 2018. Simplifying ARM concurrency: multicopy-atomic axiomatic and operational models for ARMv8. *Proc. ACM Program. Lang.* 2, POPL (2018), 19:1–19:29. <https://doi.org/10.1145/3158107>
- [19] Timothy Roscoe. 2021. It’s Time for Operating Systems to Rediscover Hardware. Keynote at OSDI’21, <https://www.usenix.org/conference/osdi21/presentation/fri-keynote>, <https://people.inf.ethz.ch/troscoe/pubs/2021-07-16-OSDIKeyNote-Handout.pdf>.
- [20] Peter Sewell, Susmit Sarkar, Scott Owens, Francesco Zappa Nardelli, and Magnus O. Myreen. 2010. x86-TSO: a rigorous and usable programmer’s model for x86 multiprocessors. *Commun. ACM* 53, 7 (2010), 89–97. <https://doi.org/10.1145/1785414.1785443>
- [21] Ben Simner, Shaked Flur, Christopher Pulte, Alasdair Armstrong, Jean Pichon-Pharabod, Luc Maranget, and Peter Sewell. 2020. ARMv8-A System Semantics: Instruction Fetch in Relaxed Architectures. In *Programming Languages and Systems - 29th European Symposium on Programming, ESOP 2020, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2020, Dublin, Ireland, April 25–30, 2020, Proceedings (Lecture Notes in Computer Science, Vol. 12075)*, Peter Müller (Ed.). Springer, 626–655. https://doi.org/10.1007/978-3-030-44914-8_23
- [22] Udo Steinberg and Bernhard Kauer. 2010. NOVA: A Microhypervisor-Based Secure Virtualization Architecture. In *Proceedings of the 5th European Conference on Computer Systems (Paris, France) (EuroSys ’10)*. Association for Computing Machinery, New York, NY, USA, 209–222. <https://doi.org/10.1145/1755913.1755935>
- [23] Steinberg, Udo and Kauer, Bernhard. 2010. Towards a Scalable Multiprocessor User-level Environment. In *Workshop on Isolation and Integration for Dependable Systems*.
- [24] The BlueRock Security Formal Methods Team. 2025. The BRiCK C++ Program Logic. <https://bluerock-io.github.io/BRiCK/>
- [25] The Coq Development Team. 2024. The Coq Reference Manual – Release 8.19.0. <https://coq.inria.fr/doc/V8.19.0/refman>.
- [26] Amin Timany, Robbert Krebbers, Derek Dreyer, and Lars Birkedal. 2024. A Logical Approach to Type Soundness. *J. ACM* 71, 6, Article 40 (Nov. 2024), 75 pages. <https://doi.org/10.1145/3676954>

- [27] Aaron Turon, Derek Dreyer, and Lars Birkedal. 2013. Unifying refinement and Hoare-style reasoning in a logic for higher-order concurrency. In *Proceedings of the 18th ACM SIGPLAN International Conference*

on Functional Programming (Boston, Massachusetts, USA) (*ICFP '13*). Association for Computing Machinery, New York, NY, USA, 377–390. <https://doi.org/10.1145/2500365.2500600>